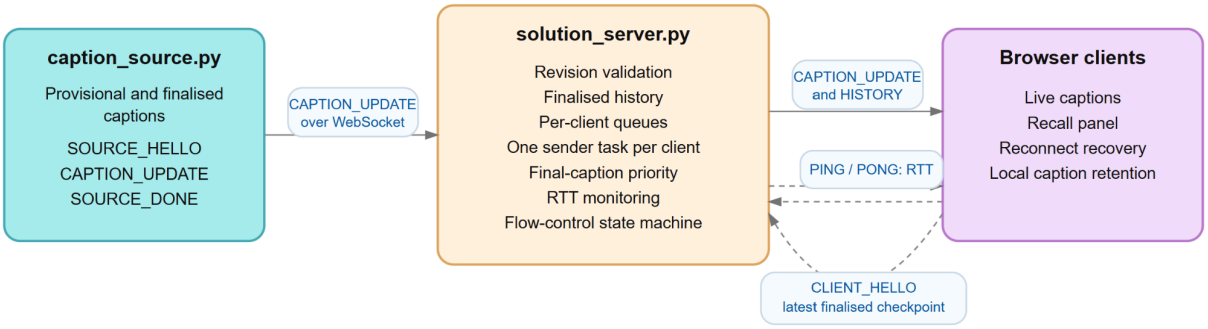
LiveRecall MiniProject Report

CS3103 Assignment 4 | Group 17

Lim Jinle, Alexius **Ong Li Min** **Tey Kai Jie** **James Chin Jia Jun**

1 Architecture and protocol



Our system follows a source–server–client architecture. caption_source.py generates provisional and finalised caption revisions and sends them to solution_server.py over WebSocket. The server validates each revision through RevisionStateManager, rejects stale or duplicate updates and stores finalised captions in bounded history for reconnect and late-join recovery.

The server maintains an independent outgoing queue and sender task for each browser client. Finalised captions are delivered in order and prioritised while provisional drafts may be replaced or temporarily held when a client falls behind. WebSocket PING/PONG exchanges provide RTT measurements for the flow-control state machine.

Browser clients send a CLIENT_HELLO containing their latest finalised segment and revision. The server uses this checkpoint to return missed captions through HISTORY before continuing with live CAPTION_UPDATE messages. On the client side, one rendered row is maintained for each segment, allowing newer revisions to replace existing rows without creating duplicates. Stale and duplicate updates are ignored while finalised captions are retained locally for the recall panel and recent-history display. This connects the server's revision and recovery mechanisms directly to the browser's live-caption rendering and recall interface.

2 Revision/recovery semantics

Each caption has a segment number, revision number and final flag. The server keeps the latest accepted caption for each segment. It rejects a duplicate or older revision. Once a segment is final, it rejects further updates to that segment. The browser applies the same checks and updates one displayed row per segment, so provisional revisions will just replace one another rather than printing duplicate lines.

When the WebSocket is connected, the browser records the latest finalized segment and revision it has received. If the WebSocket connection closes, it will reconnect automatically using exponential backoff with jitter. On reconnect it will send the finalized message in CLIENT_HELLO. The server replies with HISTORY containing later finalized captions from its recent history. The browser will then merge history caption through the above revision-checking function and prevent a replayed final caption. Locally, the browser retains recent captions for its 15/30/60 seconds recall views.

3 Backpressure Policy

When a client receives captions more slowly than the source produces them, the server must address **three** competing concerns:

1) **Outdated drafts waste client capacity.** Processing drafts that will soon be replaced can delay more important final captions.
2) **Pending captions consume server memory**. A slow client's delivery queue can grow while it catches up
3) **Drafts must remain live**. Sending only final captions would reduce traffic, but it would fail the assignment's draft caption liveness requirement.

The following is our solution to address all 3.

3.1 Detection

Every 100ms, a "monitor" (implemented as an asynchronous task, one for each client) in the solution server will send a websocket ping to each individual client, timing how long it takes the pong to respond. This RTT accounts for multiple forms of delay, but namely transmission (affected by bandwidth), queuing (segments waiting at client's receive buffer) and processing (of the client) delays will differentiate the lower RTT of a healthy client from a higher RTT of a slow / drop-prone client. In our flow_control module, we initially label any client with an RTT of above 1s to be a "potential" slow client. However, as more "on-time" probes are sent, the RTT adapts to match normal traffic conditions. For example, a 0.9s response can raise the threshold RTT to 2.7s, up to max of 3s. If there is no response within 20s, the client is disconnected and a recovery procedure is initiated. A pong counts as late when its RTT exceeds the aforementioned adaptive threshold which is based on TCP's retransmission-timeout formula [1].

```
if not late:
    # Based on RFC 6298 retransmission timer
    if self.srtt is None:
        self.srtt, self.rttvar = rtt, rtt / 2
    else:
        self.rttvar = 0.75 * self.rttvar + 0.25 * abs(self.srtt - rtt)
        self.srtt = 0.875 * self.srtt + 0.125 * rtt
    return
. . . Later in the code . . .
            # LATE_MIN = 1s, LATE_MAX = 3s
            threshold = min(max(self.srtt + 4 * self.rttvar, LATE_MIN), LATE_MAX)
```

We chose this initial 1s value from testing and, in addition, people naturally talk at about 196 WPM or about approx 3.3 words per second [2], which is a comfortable pace for draft captions (and thereby, if RTT overshoots, we are behind pace and should engage backpressure). Note that this latency-ping test is also described in Python's websocket's documentation, which also states how this latency can arise due to bad network conditions [3].

3.2 Response

Each client connection has its own state machine, with clients starting out at the **NORMAL** state.

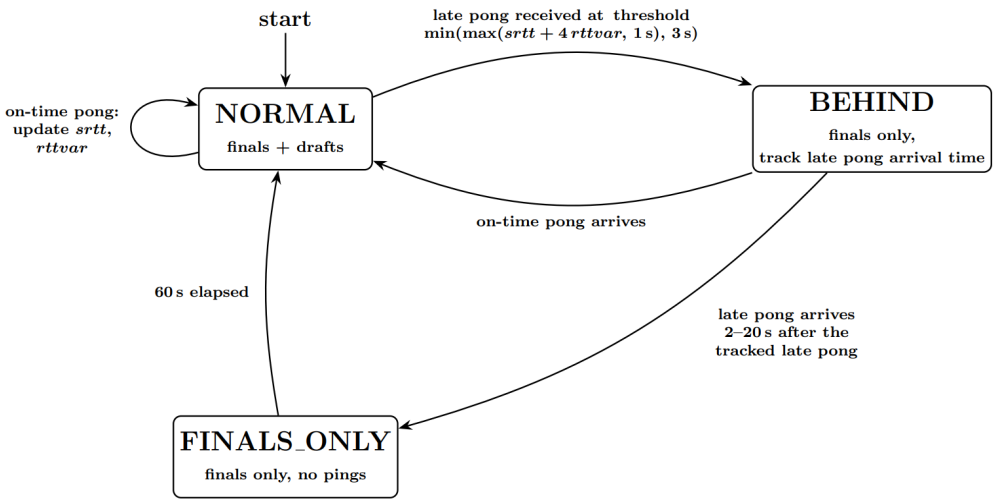
NORMAL: The server sends drafts and finals as quickly as it can write to the send buffer. If an update to the draft is received before the draft is sent out, the draft will be updated. If the server receives more finals than it can send, finals are added to a client specific bounded queue (max 256 finals, overfill restarts the connection and utilizes reconnect recovery). During NORMAL, if a late pong crossing the RTT threshold arrives, the state for that connection temporarily changes to BEHIND.

BEHIND: Drafts are withheld from being sent. Finals will be continued to be sent as per normal. The objective is to stop pressuring the slow client's receive buffer with drafts which we estimate will not be processed in time to be "relevant". This allows the client to clear up its backlog. At this juncture, if the late pong that caused the

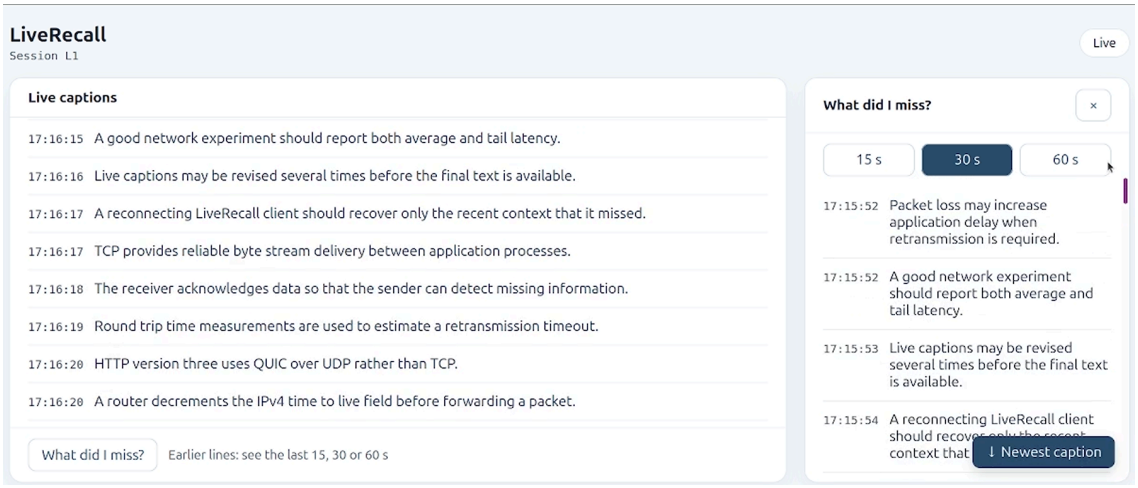
NORMAL -> BEHIND transition is received within 20s (see recovery if >20s), we mark the time it was received and keep track of the next few pongs:

- On next on-time pong -> **NORMAL** (but keep track of the late pong timestamp)
- If late pong is received within the next 2s, treat them as part of a possible outlier burst -> **BEHIND**
- If late pong is received within 2-20s of the tracked late pong -> **BEHIND** -> **FINALS_ONLY**
- If late pong is received after the 20s, replace the last tracked late pong -> **BEHIND**

FINALS_ONLY: Similar to BEHIND, drafts will be withheld and only finals will be sent. Additionally, FINALS_ONLY will start a 60s timer where no pings will be sent, further alleviating the backpressure. Once the 60s is finished, it transits back to NORMAL to attempt a recovery.



4 User Interface



5 Public Experiments

Each public experiment was run three times using the provided deterministic test harness. The results table reports the median across the three independent runs.

5.1 Results

Table A1 gives the median of three runs for each test. Without impairment, the system is fast and complete: with 20 clients (T1) and 100 clients (T2) the P50 is 3.09 ms and 8.73 ms and the P99 is under 20 ms, with 100% fresh captions and 100% final completeness. The latency grows roughly in proportion to the number of clients, since each update is sent to every client. T4 has the same latency, and its fresh-final ratio is slightly below 100%

because finals missed during the disconnect arrive after reconnection. In T3, the slow 10% of clients set the P95 and P99 tails and lower the fresh ratios, while the median stays low. Final completeness is still 100% in all tests. In T5, the injected delay and loss raise latency (P50 263.91 ms). Final completeness stays at 100% as TCP retransmits lost packets, so loss appears as delay, and the server never drops finalised captions.

6 Flow Control Optimisation Comparison for T3

To illustrate the effectiveness of the flow control optimization, we compare T3 before and after the flow control was added as it is the public test with slow clients which is the case flow control is designed to handle. In the other tests, every client keeps up, so flow control never activates. Table A2 compares T3 before and after flow control.

Without flow control, a slow client's backlog builds up in buffers while ws.send() returns immediately, so the server gets no sign that the client is behind. With flow control, the ping/pong probe exposes the lag and the server holds that client's drafts so its backlog drains. Overall P95 falls from 2,352.18 ms to 497.70 ms and P99 from 235,327.96 ms to 11,347.98 ms, and final completeness rises from 91.45% to 100%.

Splitting by client type (Table A3 & A4) shows where the gain comes from. Slow clients' median caption age halves (14,421 ms to 6,923 ms) and their P95 falls from 28,082 ms to 13,377 ms. Healthy clients are not penalised: their P50 is essentially unchanged (8.28 ms to 8.94 ms) and their P95 improves from 37.31 ms to 16.23 ms. Slow clients are still seconds behind because lag is only detected after their queue has filled (about 16 s).

7 Failures & Trade-off

It should be noted that, due to restrictions imposed because of desired features, under certain conditions, our application will be unable to achieve certain results.

7.1 Bounded Queues and Completeness

The advantage of having a Bounded Queue, as required by the assignment brief, is that the queues do not grow indefinitely and crash the application due to memory exhaustion. Growth of the queue usually arises when the throughput to a client (from server) is consistently and/or significantly lower than the throughput from source (to server).

Eventually, if the source has too many captions and the slow client does not catch up, the queue will fill up to its bounds and cannot take in any more segments. The additional segment cannot be stored as that would defeat the purpose of bounding, so a segment which the client is missing will likely be lost. This means that completeness and accuracy of the client’s captioning cannot be guaranteed for slow clients. In this assignment, we assumed that the number of final segments sent by the source is not too large and that revision coalescing and large bounds, with queues of 256 segments, would be enough to accommodate a slow client. However, we acknowledge that the above issue may present itself in a real-world test run.

8. Limitations

The server stores finalised-caption history in memory and retains only a bounded recent window. Captions outside this window are therefore unavailable through reconnect or late-join recovery. The browser also keeps recall content locally for the current session rather than providing permanent transcript storage.

The current implementation supports one active caption source per lecture session. Supporting multiple simultaneous sessions would require separate session-level revision managers histories and client subscriptions which are outside the scope of this project.

The flow-control state machine uses RTT measurements and repeated delayed responses to estimate client responsiveness. These measurements may be affected by network conditions, browser scheduling and operating-system activity. Therefore latency and completeness results may vary slightly between runs and should be interpreted in the context of the tested environment.

9. Conclusion

LiveRecall provides an end-to-end caption delivery system supporting provisional revisions, finalised captions, reconnect, recovery, late joining and slow-client handling. The RevisionStateManager ensures that newer revisions replace older ones while stale and duplicate updates are ignored. Bounded finalised-caption history allows clients to recover missed content after reconnecting.

The server uses an independent queue and sender task for each browser client. Finalised captions are delivered in order and prioritised while provisional drafts can be replaced or temporarily held when necessary. This allows each client to be served independently without interrupting the caption source or other connected clients.

The browser client maintains one rendered row per segment and applies newer revisions in place. It also preserves recent finalised captions for recall and displays reconnect recovery feedback. Together these components provide a practical balance between caption freshness finalised-caption completeness recovery support and resource control.

10. Future Work

Future development could extend the system with configurable retention periods and recovery policies to support a wider range of lecture and training scenarios. Persistent transcript storage could also allow finalised captions to remain available for later review across multiple sessions.

Additional testing could examine longer sessions, larger client populations and deployment configurations beyond those included in the supplied experiments. This would provide further evidence of the system's behaviour across a wider range of usage scenarios while preserving its current architecture and functionality.

Appendix

(Note all tests were ran within a Ubuntu VM)

Table A1: Public Test Case Results (Final Iteration)

Test	Scenario	Clients	P50 (ms)	P95 (ms)	P99 (ms)	Fresh Caption Ratio (%)	Freshness Final Caption Ratio (%)	Normal Update Coverage (%)	Final Completeness (%)
T1	Basic correctness	20	3.09	4.39	6.21	100.00	100.00	100	100.00
T2	Public scale	100	8.73	15.18	19.72	100.00	100.00	100	100.00
T3	Slow clients	100	9.25	497.70	11347.98	95.01	90.53	95	100.00
T4	Reconnect	100	7.03	15.32	20.66	100.00	98.42	98.42	100.00
T5	Network Impairment	100	263.91	594.06	626.76	1030.01	86.02	93.03	100.00

Table A2: Overall T3 Performance before and after flow-control implementation

Version	Clients	P50 (ms)	P95 (ms)	P99 (ms)	Fresh Caption Ratio (%)	Fresh Final Caption Ratio (%)	Normal Update Coverage (%)	Final Completeness (%)
Before Flow Control	100	8.57	2352.18	235327.96	94.63	86.45	91.45	91.45
After Flow Control added	100	9.25	497.70	11347.98	95.01	90.53	95	100.00

Table A3: Normal-client performance in T3 before and after flow control

Version	P50 (ms)	P95 (ms)	Fresh (%)
Before Flow Control	8.28	37.31	100.0
After Flow Control	8.94	16.23	100.0

Table A4: Slow-client performance in T3 before and after flow control

Version	P50 (ms)	P95 (ms)	Fresh (%)
Before Flow Control	14421.04	28082.20	1.75
After Flow Control	6923.07	13377.22	5.26

References

[1] V. Paxson, M. Allman, J. Chu, and M. Sargent, “Computing TCP’s Retransmission Timer,” June 2011, doi: 10.17487/rfc6298.
[2] J. Yuan, M. Liberman, and C. Cieri, “Towards an Integrated Understanding of Speaking Rate in Conversation,” in *INTERSPEECH 2006 - ICSLP*, Pittsburgh, Pennsylvania, Sep. 2006, pp. 541–544. doi: 10.21437/Interspeech.2006-204.
[3] Aymeric Augustin, “Keepalive and Latency,” websockets, 2026. https://websockets.readthedocs.io/en/stable/topics/keepalive.html (accessed Nov. 02, 2026).